Section 1

Управление уязвимостями

От списка CVE к работающей системе

Курс «Защита операционных систем»

План лекции

1. Почему это отдельная дисциплина
2. CVE, CVSS, EPSS, KEV: что чем отвечает
3. Жизненный цикл управления уязвимостями
4. Инвентаризация и обнаружение в Linux
5. Приоритизация: почему не по CVSS
6. Компенсирующие меры и патчи
7. Кейсы: CUPS и Dirty Pipe
8. Проверка, аудит, журналирование
9. Переход к лабораторной

Почему отдельная дисциплина

- Безопасность приложений отвечает на вопрос «как написано ПО».
- Эксплуатация — «как это ломают».
- **Управление уязвимостями** — «что делать с уже известными дефектами в уже установленном ПО».
- Это процесс администрирования: инвентаризация, версии, сроки, простой, регресс.
- Практический результат измеряется не числом закрытых CVE, а **снижением экспозиции**.

Словарь

Понятие	Что означает	Чего не означает
CVE	идентификатор уязвимости	не говорит, насколько она опасна
CVSS	техническая тяжесть 0–10	не вероятность эксплуатации
EPSS	вероятность эксплуатации в ближайшие 30 дней	не тяжесть последствий
KEV	список уже эксплуатируемых уязвимостей	не полноту списка
SCA / SBOM	что именно установлено	не факт эксплуатации

Жизненный цикл

1. **Инвентаризация** — что установлено, каких версий, где работает
2. **Обнаружение** — соотнесение версий с известными CVE
3. **Приоритизация** — что и в каком порядке исправлять
4. **Устранение** — патч или компенсирующая мера
5. **Проверка** — доказательство, что риск закрыт (и функция не сломана)

Пропущенный пятый шаг — самая частая ошибка: «обновили» ≠ «закрыли».

Инвентаризация в Linux

- Пакеты: `dpkg -l`, `rpm -qa`, `apt-cache policy <пакет>`
- Обновления: `apt list --upgradable`
- Ядро: `uname -r` (версия ядра — отдельный актив, контейнеры его не меняют)
- Контейнеры: `docker image ls`, теги образов, SBOM
- Что важно: **точная версия**, а не «последняя»

```
dpkg-query -W -f='${Package} ${Version}\n' cups cups-browsed
apt-cache policy cups-browsed | head -5
```

Обнаружение

- Локальные средства: debsecan, apt + сравнение версий
- Сканеры: OpenVAS/Greenbone, Trivy (образы), Lynis (аудит)
- SIEM/EDR: Wazuh **Vulnerability Detector** — сверяет инвентарь агента с базой CVE
- Ограничение любого сканера: он видит **версию**, а не конфигурацию
- Поэтому обнаружение всегда дополняется проверкой экспозиции:

```
ss -ulpn | grep ':631'      # слушает ли служба внешние интерфейсы
```

Приоритизация: не по CVSS

Практическая формула:

приоритет = тяжесть × вероятность эксплуатации × ценность актива ÷ стоимость меры

Что учитывать:

1. **Экспозиция**: `0.0.0.0` или `127.0.0.1`; есть ли путь из недоверенной сети
2. **Условия эксплуатации**: нужна ли аутентификация, действие пользователя
3. **Доступность фикса**: есть ли патч, нужен ли простой/перезагрузка
4. **Ценность актива**: что именно теряем при компрометации

CVSS 9.8 на изолированном стенде может ждать месяц; CVSS 7.8 на публичном сервере — нет.

Компенсирующие меры

Быстрее патча и часто достаточны на время:

- отключить ненужную службу (`systemctl disable --now cups-browsed`);
- сузить привязку: `127.0.0.1` вместо `0.0.0.0`;
- закрыть порт firewall (nftables, allowlist);
- сегментировать сеть; убрать публикацию порта контейнера.

Патч закрывает **причину**. Компенсирующая мера закрывает **путь**. Правильная практика — сначала мера (минуты), затем патч (часы/дни), затем проверка обоих.

Кейс: CUPS (2024)

Цепочка из четырёх CVE:

1. **CVE-2024-47176** — `cups-browsed` слушает UDP/631 и доверяет анонсам от любого источника
2. **CVE-2024-47076** — IPP-атрибуты от «принтера» не проверяются
3. **CVE-2024-47175** — непроверенные данные попадают в PPD-файл
4. **CVE-2024-47177** — директива `FoomaticRIPCommandLine` даёт выполнение команды

Итог: удалённое выполнение кода, но при печати на подставной принтер — то есть нужна ещё и действие пользователя.

Кейс CUPS: что показательно

- Уязвим не «Linux», а **конкретные версии** `cups-browsed`, `libcupsfilters`, `libppd`, `cups-filters`.
- Оценки CVSS 8.4–9.1 при реальном ограничении: нужен print job.
- Служба включается по умолчанию во многих дистрибутивах — «просто работает» и есть риск.
- Меры: обновить пакеты, отключить `cups-browsed`, `BrowseRemoteProtocols none`, закрыть UDP/631.
- В актуальных Debian/Ubuntu уязвимых версий уже нет — воспроизведение требует зафиксированных версий (контейнер).

Кейс: Dirty Pipe (2022)

- **CVE-2022-0847**, ядро 5.8–5.16.10, локальное повышение привилегий до root.
- Контейнер **не помогает**: ядро общее с хостом.
- Единственная мера — обновление ядра и перезагрузка.
- Проверка: `uname -r` и сравнение с 5.10.102 / 5.15.25 / 5.16.11.
- Вывод: уязвимости ядра — отдельный класс актива со своим циклом (нужен ребут, нужен план обслуживания).

Проверка после устранения

Три уровня доказательств:

1. **Состояние**: `check_vuln.sh` → 0 признаков; `ss` не показывает порт
2. **Работает ли мера**: счётчики nftables растут при попытке подключения
3. **Не сломали ли функцию**: `lpstat -r`, тестовая печать, бизнес-операции

И отдельно — **персистентность**: перезагрузка/перезапуск контейнера не должны возвращать уязвимость.

Аудит и журналирование

Без телеметрии нет расследования:

- firewall: `nft ... log prefix "CUPS-DROP: "` → `journalctl -k`
- службы: `journalctl -u cups-browsed`, `/var/log/cups/*`
- изменения: `auditd` (правила на файлы конфигурации), `dpkg/apt history`
- централизация: Wazuh Agent → Manager → Dashboard

Что искать: обращения к закрытому порту, запуск отключённой службы, изменение `cups-browsed.conf` вне окна обновлений.

Wazuh: две роли

1. **Vulnerability Detector** — инвентарь агента + база CVE = список уязвимостей по хостам
2. **Правила и алерты** — превращают событие в сигнал

```
<rule id="100631" level="7">
  <match>CUPS-DROP</match>
  <description>CUPS: обращение к UDP/631 заблокировано firewall</des
</rule>
```

Проверка правила — `wazuh-logtest`, до того как оно попадёт в продакшн. Ограничение: детект по логу видит только то, что мы решили логировать.

Итоги

- Уязвимость — свойство **версии**, а не «системы вообще».
- Приоритет считается по экспозиции и ценности актива, а не только по CVSS.
- Компенсирующая мера и патч дополняют друг друга.
- Проверка обязательна: состояние, работа меры, отсутствие регресса, персистентность.
- Логи — мост от защиты к расследованию инцидентов.

Лабораторная: обнаружение → приоритизация → устранение → проверка на стенде CUPS.